ovre

SMART CONTRACT AUDIT

ZOKYO.

Oct 27th, 2021 | v. 1.0

PASS

Zokyo Security Team has concluded that this smart contract passes security qualifications. The contracts can be deployed on the mainnet.

TECHNICAL SUMMARY

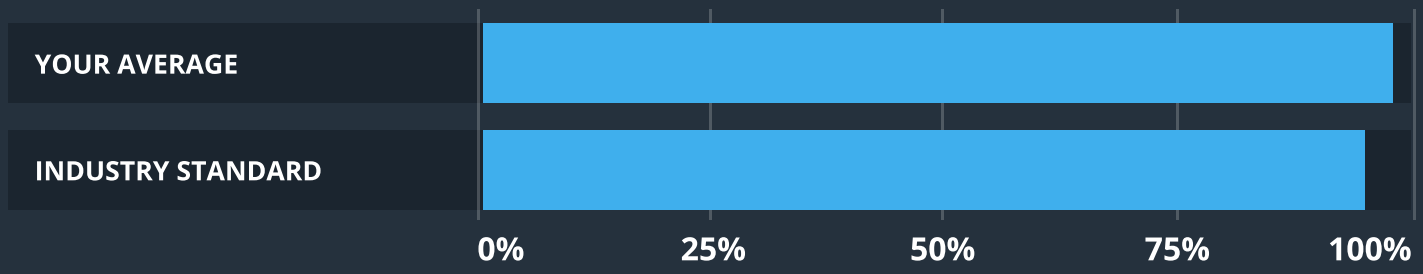
This document outlines the overall security of the ZooCoin smart contracts, evaluated by Zokyo's Blockchain Security team.

The scope of this audit was to analyze and document the ZooCoin smart contract codebase for quality, security, and correctness.

Contract Status

There were no critical issues found during the audit.

Testable Code



The testable code is 98%, which is above the industry standard of 95%.

It should be noted that this audit is not an endorsement of the reliability or effectiveness of the contract, rather limited to an assessment of the logic and implementation. In order to ensure a secure contract that's able to withstand the Ethereum network's fast-paced and rapidly changing environment, we at Zokyo recommend that the ZooCoin team put in place a bug bounty program to encourage further and active analysis of the smart contract.

TABLE OF CONTENTS

Auditing Strategy and Techniques Applied 3
Summary 5
Structure and Organization of Document 6
Complete Analysis 7
Code Coverage and Test Results for all files 15
Tests written by Zokyo Secured team 15

AUDITING STRATEGY AND TECHNIQUES APPLIED

The Smart contract's source code was taken from the ZooCoin repository.

Repository:
https://github.com/ZooEcosystem/Ovre/commit/f7f63d7b2a72a21194d473b35c82d19d4a54d92b

Last commit:
fefbfc857140ad8f71b1503b330bf349fe3c63a8

Functions under the scope:

- AuctionFactory;
- ERC721Upgradeable;
- InviteAFriendSystem.

Throughout the review process, care was taken to ensure that the token contract:

- Implements and adheres to existing Token standards appropriately and effectively;
- Documentation and code comments match logic and behavior;
- Distributes tokens in a manner that matches calculations;
- Follows best practices in efficient use of gas, without unnecessary waste;
- Uses methods safe from reentrance attacks;
- Is not affected by the latest vulnerabilities;
- Whether the code meets best practices in code readability, etc.

Zokyo's Security Team has followed best practices and industry-standard techniques to verify the implementation of ZooCoin smart contracts. To do so, the code is reviewed line-by-line by our smart contract developers, documenting any issues as they are discovered. Part of this work includes writing a unit test suite using the Truffle testing framework. In summary, our strategies consist largely of manual collaboration between multiple team members at each stage of the review:

1. Due diligence in assessing the overall code quality of the codebase.
2. Cross-comparison with other, similar smart contracts by industry leaders.
3. Testing contract logic against common and uncommon attack vectors.
4. Thorough, manual review of the codebase, line-by-line.

SUMMARY

Zokyo team has concluded a security audit for the given smart contracts. During the process, a couple of issues were found. All of the mentioned issues are described below in the Compete Analysis section.

It is worth mentioning that there were no critical issues found. All the mentioned findings may have an effect only in the case of specific conditions performed by the contract owner. Mostly all of the recommendations were taken into consideration. Hence, all the severe issues were fixed and only a couple of informational ones were left unresolved.

Based on the findings during the audit and on the fixes provided by ZooCoin team, we can state that the contracts are safe for the end-user and the contract owner. The given codebase can be deployed to the mainnet.

The overall score of the audited smart contracts is 98.

STRUCTURE AND ORGANIZATION OF DOCUMENT

For ease of navigation, sections are arranged from most critical to least critical. Issues are tagged “Resolved” or “Unresolved” depending on whether they have been fixed or addressed. Furthermore, the severity of each issue is written as assessed by the risk of exploitation or other unexpected or otherwise unsafe behavior:

Critical

The issue affects the ability of the contract to compile or operate in a significant way.

High

The issue affects the ability of the contract to compile or operate in a significant way.

Medium

The issue affects the ability of the contract to operate in a way that doesn’t significantly hinder its behavior.

Low

The issue has minimal impact on the contract’s ability to operate.

Informational

The issue has no impact on the contract’s ability to operate.

COMPLETE ANALYSIS

Duplicating of interfaces name

HIGH | RESOLVED

Compiler return this error:

```
> Duplicate contract names found for InviteAFriendSystem.
> This can cause errors and unknown behavior. Please rename one of your contracts.
```

because you have 2 interfaces with the same name “InviteAFriendSystem” in AuctionFactory.sol and ERC721Upgradeable.sol:

```
interface InviteAFriendSystem {
    function addInvite(address artist) external;
}
```

```
interface InviteAFriendSystem {
    function isArtistApproved(address artist) external view returns(bool);
    function invites(address) external view returns (bool);
    function useInvite(address artist) external;
}
```

Recommendation:
Change one of the interfaces or remove them from AuctionFactory.sol and ERC721Upgradeable.sol and add as a separate file.

Re-audit:
Fixed.

Wrong path of import Ownable.sol the in InviteAFriendSystem.sol

HIGH | RESOLVED

True path of import Ownable.sol is "@openzeppelin/contracts/access/Ownable.sol"

```
// SPDX-License-Identifier: UNLICENSED
pragma solidity ^0.8.9;

import "@openzeppelin/contracts/Ownable.sol";
```

Recommendation:
Change import path from “@openzeppelin/contracts/Ownable.sol” to “@openzeppelin/contracts/access/Ownable.sol”

Re-audit:
Fixed.

The owner has too many permissions

MEDIUM | RESOLVED

An owner of the OVRE contract has permission to change the fee at any time. It means that the owner can set a fee of up to 100% and potential users would be forced to pay this fee.

Recommendation:
There are a couple of solutions:

- Announce newFee in the contract so users will know if the fee will change and new fee value;
- Remove the setter for the fee and make the fee immutable.

Re-audit:
Fixed.

Use of deprecated transfer

LOW | RESOLVED

The EnglishAuctionReservePrice contract uses a transfer method to transfer currency to users. Since transfers have fixed 2300 gas for executing it is unsafe to use it with new versions of solidity.

Recommendation:
Use call instead of transfer.

Re-audit:
Fixed. Use call instead of transfer.

Two contracts in the same file

LOW | RESOLVED

EnglishAuctionReservePrice and AuctionFactory are in one file. It means that one file contains logic for two separate contracts. To avoid this you can use interfaces instead.

Recommendation:
Separate contracts. Use interfaces.

Re-audit:
Fixed.

Contracts do not pass lint

INFORMATIONAL | UNRESOLVED

It's good to keep your code nice and clean. Ethlint found 50 warnings and 3 errors.

Recommendation:
Fix ethlint issues or use comments to ignore linting.

Re-audit:
Skipped.

Gas saving for the OVRE contract

INFORMATIONAL | RESOLVED

The OVRE contract uses the safeMint function to mint new tokens for users. This function is called only externally. To avoid extra gas usage it can be marked as external instead of public.

Recommendation:
Change accessibility of the safeMint function to external.

Re-audit:
Fixed.

Different versions of contracts

INFORMATIONAL | RESOLVED

The OVRE contract uses >0.8.2 version of solidity everything else using >0.8.7. Use a similar solidity version for all contracts in the project to make a solid structure.

Recommendation:
Change the version of the OVRE contract to >0.8.7.

Re-audit:
Fixed.

Duplicated functions

INFORMATIONAL | RESOLVED

The InvateAFriendSystem contract has an invite variable to define users who were invited. This variable is public which means that solidity will make getter for it but the contract already has the hasInvite function.

Recommendation:
Make invites internal/private.

Re-audit:
Fixed.

Not used variables

INFORMATIONAL | RESOLVED

In the EnglishAuctionReservePrice there is unused internal variable inviteAFriend.

Recommendation:
Remove unused variable.

Re-audit:
Fixed.

	AuctionFactory	ERC721Upgradeable
Re-entrancy	Pass	Pass
Access Management Hierarchy	Pass	Pass
Arithmetic Over/Under Flows	Pass	Pass
Unexpected Ether	Pass	Pass
Delegatecall	Pass	Pass
Default Public Visibility	Pass	Pass
Hidden Malicious Code	Pass	Pass
Entropy Illusion (Lack of Randomness)	Pass	Pass
External Contract Referencing	Pass	Pass
Short Address/ Parameter Attack	Pass	Pass
Unchecked CALL Return Values	Pass	Pass
Race Conditions / Front Running	Pass	Pass
General Denial Of Service (DOS)	Pass	Pass
Uninitialized Storage Pointers	Pass	Pass
Floating Points and Precision	Pass	Pass
Tx.Origin Authentication	Pass	Pass
Signatures Replay	Pass	Pass
Pool Asset Security (backdoors in the underlying ERC-20)	Pass	Pass

	InviteAFriendSystem
Re-entrancy	Pass
Access Management Hierarchy	Pass
Arithmetic Over/Under Flows	Pass
Unexpected Ether	Pass
Delegatecall	Pass
Default Public Visibility	Pass
Hidden Malicious Code	Pass
Entropy Illusion (Lack of Randomness)	Pass
External Contract Referencing	Pass
Short Address/ Parameter Attack	Pass
Unchecked CALL Return Values	Pass
Race Conditions / Front Running	Pass
General Denial Of Service (DOS)	Pass
Uninitialized Storage Pointers	Pass
Floating Points and Precision	Pass
Tx.Origin Authentication	Pass
Signatures Replay	Pass
Pool Asset Security (backdoors in the underlying ERC-20)	Pass

CODE COVERAGE AND TEST RESULTS FOR ALL FILES

Tests written by Zokyo Security team

As part of our work assisting ZooCoin in verifying the correctness of their contract code, our team was responsible for writing integration tests using the Truffle testing framework.

Tests were based on the functionality of the code, as well as a review of the ZooCoin contract requirements for details about issuance amounts and how the system handles these.

Code Coverage

The resulting code coverage (i.e., the ratio of tests-to-code) is as follows:

FILE	% STMTS	% BRANCH	% FUNCS	% LINES
contracts/	100.00	96.55	100.00	100.00
IERC20.sol	100.00	95.24	100.00	100.00
IERC20Metadat.sol	100.00	100.00	100.00	100.00
JustYours.sol	100.00	100.00	100.00	100.00
IERC20Metadat.sol	100.00	100.00	100.00	100.00
All files	**100.00**	**96.55**	**100.00**	**100.00**

Test Results

```
Contract: AuctionFactory & EnglishAuctionReservePrice
  Methods of AuctionFactory
    check for correct deployment
      ✓ Must return correct deployer address (128ms)
    get and set inviteAFriendSystemAddress
      ✓ Must return correct value (95ms)
```

```
      ✓ Must fail to set a new value of inviteAFriendSystemAddress if a caller isn't owner (212ms)
      ✓ Must set new value (379ms)
    get and set FeeCollector
      ✓ Must return correct value (106ms)
      ✓ Must fail to set a new value of FeeCollector if a caller isn't owner (210ms)
      ✓ Must set new value (399ms)
    get and set step
      ✓ Must return correct value (74ms)
      ✓ Must fail to set a new value of step if a caller isn't owner (194ms)
      ✓ Must set new value (517ms)
    get and set NFTAddress
      ✓ Must return correct value (114ms)
      ✓ Must fail to set a new value of NFTAddress if a caller isn't owner (203ms)
      ✓ Must set new value (510ms)
    get and set ZooFeePercentage
      ✓ Must return correct value (90ms)
      ✓ Must fail to set a new value of ZooFeePercentage if a caller isn't owner (193ms)
      ✓ Must fail to set a new value of ZooFeePercentage if new fee value is more than 15 (259ms)
      ✓ Must set new value (426ms)
    createAuction
      ✓ Must fail if a caller isn't an owner of the token (270ms)
      ✓ Must work correctly (474ms)
    getAuctionDetailsByTokenId
      ✓ Must fail if Auction with such a token id doesn't exist (113ms)
      ✓ Must return correct data about Auction (181ms)
    addInviteToArtist
      ✓ Must fail if caller isn't SpawnedAuction (180ms)
    fireEvent
      ✓ Must fail if caller isn't SpawnedAuction (241ms)
  Methods of EnglishAuctionReservePrice
    bid
      ✓ Must fail if block.timestamp >= minimumStartTime (287ms)
      ✓ Must fail if value for bid is less than lastBid(_reservePriceWei) (220ms)
      ✓ Must fail if value for bid is less than lastBid (519ms)
      ✓ Must fail if auction has already finished (834ms)
      ✓ Must fail if caller is another contract (523ms)
    need to finish check event
      ✓ Must make first bid correctly (777ms)
```

```
        ✓ Must work correctly if one user bids two times (850ms)
        ✓ Must fail if one user bids two times with the same value (890ms)
        ✓ Must make the second bid correctly (1726ms)
        ✓ Must rise endTime for 5 minutes (2030ms)
    pull
        ✓ Must fail if caller isn't controller (295ms)
        ✓ Must fail if auction pull has already finished (1172ms)
        ✓ Must fail if some user has already bid (605ms)
        ✓ Must work correctly (1096ms)
    adminPull
        ✓ Must fail if caller isn't feeCollector (292ms)
        ✓ Must fail if auction pull has already finished (2347ms)
        ✓ Must fail if some user has already bid (716ms)
        ✓ Must work correctly (1293ms)
    changeReservePrice
        ✓ Must fail if caller isn't controller (411ms)
        ✓ Must fail if auction pull has already finished (2319ms)
        ✓ Must fail if some user has already bid (877ms)
        ✓ Must work correctly (1519ms)
    end
        ✓ Must fail if this function has been already called (1769ms)
        ✓ Must fail if contract hasn't bids (235ms)
        ✓ Must fail if contract is living now (710ms)
        ✓ Must work correctly (2656ms)

Contract: OVRE
  Methods
    constructor
        ✓ Check for correct deploy (956ms)
    safeMint
        ✓ Must fail if caller isn't approved (226ms)
        ✓ Must work correctly (1198ms)
    changeFee
        ✓ Must fail if caller isn't owner (202ms)
        ✓ Must work correctly (813ms)
    changeFeeDenominator
        ✓ Must fail if caller isn't owner (229ms)
        ✓ Must work correctly (492ms)
```

```
    burn
      ✓ Must fail if token doesn't exist (860ms)
      ✓ Must fail if caller isn't approved (250ms)
      ✓ Must work correctly (1602ms)
    supportsInterface
      ✓ Must return true (369ms)
      ✓ Must return false (223ms)

Contract: AccessControl
  Methods
    constructor
      ✓ Check for correct deploy (124ms)
    addApprovedArtist
      ✓ Must fail if caller isn't owner (209ms)
      ✓ Must work correctly (993ms)
    removeApprovedArtist
      ✓ Must fail if caller isn't owner (198ms)
      ✓ Must work correctly (1615ms)
    setAuctionFactory
      ✓ Must fail if caller isn't owner (201ms)
      ✓ Must work correctly (864ms)
    setNFTContract
      ✓ Must fail if caller isn't owner (332ms)
      ✓ Must work correctly (186ms)
    addInvite
      ✓ Must fail if caller isn't AuctionFactory (293ms)
      ✓ Must add 2 invites for one artist (418ms)
      ✓ Must work correctly (320ms)
    inviteAFriend
      ✓ Must fail if caller doesn’t have an invite (198ms)
      ✓ Must fail if caller has already used his invite (1428ms)
      ✓ Must work correctly (996ms)

77 passing (1m)
```

We are grateful to have been given the opportunity to work with the ZooCoin team.

The statements made in this document should not be interpreted as investment or legal advice, nor should its authors be held accountable for decisions made based on them.

Zokyo's Security Team recommends that the ZooCoin team put in place a bug bounty program to encourage further analysis of the smart contract by third parties.

ZOKYO.